A Resilient Runtime-Verification Fabric for Security Monitoring of Critical Edge-IoT Infrastructure

Nikolaos Kekatos (✉)[1], Marinelio Chintri[2], Panagiotis Katsaros[3], Alexios Lekidis[4], Tom Nianios[1], Ioannis Seitoglou[2], Anastasios Temperekidis[1], and Stylianos Basagiannis[2]

[1]Clone Systems, Cyprus, {nkekatos,tnianios,atemperekidis}@clone-systems.com
[2]International Hellenic University, Greece,
{basagiannis,marchint,ioaseito}@ihu.gr
[3]Aristotle University of Thessaloniki, Greece, katsaros@csd.auth.gr
[4]University of Thessaly, Greece, alekidis@uth.gr
(✉) Corresponding author.

Abstract. Protecting critical infrastructure increasingly depends on continuously verifying large IoT fleets against formal security specifications at runtime. Yet the runtime-verification (RV) pipelines proposed for this task are typically single-host prototypes whose monitors read a shared log file, an architecture with no resilience to the failures such deployments incur: a crash or overload silently drops events, clock skew corrupts the ordering that metric temporal monitors require, a time-triggered "node has gone silent" property cannot fire when the network itself falls silent, and one slow consumer stalls the pipeline. Each failure is *silent*: the monitor keeps emitting verdicts over a corrupted view. We present RV-Fabric, a resilient delivery layer that carries the hierarchy over two brokers (MQTT for device ingest, a durable stream broker for backend delivery) and re-establishes five continuity guarantees: durable delivery under post-persistence crashes, a trusted event order, progress under total silence, consumer isolation, and flow control under bounded overload, each holding as an invariant conditioned on broker durability. Above the transport, RV-Fabric makes evidence completeness part of runtime-verification semantics: every verdict carries a status (sound, degraded, incomplete, or unavailable) derived from delivery gaps, retention pressure, and liveness, so an incomplete stream can no longer yield an unqualified all-clear. We evaluate the guarantees under controlled fault injection (crashes, clock skew, network silence, and overload) on a containerised testbed. Measured against a fault-free oracle using the real MonPoly engine and tick-driven liveness detectors, the shared-log baseline misses six of seven injected security incidents and reports each as an unqualified all-clear, whereas RV-Fabric preserves all seven, while removing individual mechanisms reintroduces specific classes of loss; two openly published critical-infrastructure datasets (water-SCADA and IoT/IIoT) further replay end-to-end through the real engine.

Keywords: Critical infrastructure protection · Resilience · Runtime verification · IoT security · Intrusion detection · Situational awareness · Edge-cloud continuum

1 Introduction

Security monitoring for the Internet of Things is moving off individual devices onto the tiered edge-cloud continuum, where evidence from many devices must be collected, ordered, and reasoned about beyond any single node [24]. This matters because large IoT fleets are now both critical infrastructure and a favoured attack surface: internet-facing devices are routinely compromised and conscripted into coordinated campaigns that a single device, seeing only its own traffic, cannot detect [25,10]. Runtime verification (RV), which continuously checks a running system against explicit formal specifications [18,3], suits such behaviour: unlike signature- or learning-based intrusion detection, an RV monitor flags any execution that violates a stated property and emits an auditable witness. Recent hierarchical designs push this across a deployment's tiers, with lightweight per-device checks, per-gateway aggregation, and fleet-wide temporal-logic correlation catching coordinated cross-device attacks no single node can see [15,14].

The verification logic of these designs is mature, but the delivery layer beneath them is not, and for critical infrastructure that layer is where resilience is won or lost. Published prototypes run on a single host, with every monitor reading from one shared append-only log file. That arrangement suits a formal-methods evaluation but collapses on the edge-cloud continuum: *(i)* device-to-gateway links are lossy, so records vanish before a monitor ever sees them; *(ii)* devices and gateways keep independent, skewing clocks, breaking the monotone-timestamp assumption metric temporal-logic monitors depend on; *(iii)* time-triggered properties ("no device has reported for T seconds") never fire when the network goes silent, because a log with no new lines produces no events to advance the monitor's clock; and *(iv)* a single shared log serialises all consumers with no flow control, so one slow monitor stalls the pipeline. These failures are silent: the monitors keep emitting verdicts from a corrupted view of the fleet, so a dropped or mis-ordered security-relevant event may cause an attack to go unflagged, exactly the continuity failure a critical-infrastructure operator cannot tolerate (Fig. 1). *In a distributed deployment, no news is not good news: the completeness of the evidence stream has to be carried inside the verdict, not assumed behind it.*

Contributions. This paper makes four contributions. *(1)* RV-Fabric, a resilient delivery layer carrying an unchanged three-layer monitoring hierarchy [15,14] across the edge-cloud continuum, re-establishing the delivery, ordering, liveness, isolation, and flow-control guarantees a shared log cannot provide (Sect. 3). *(2)* A threat model for verification-grade monitoring mapping a tiered network/device/gateway adversary to its mitigation, with the five continuity properties stated as broker-contingent invariants (Sects. 4–5).

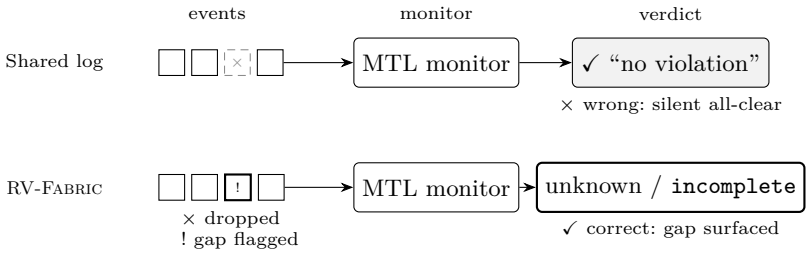

Fig. 1. The failure mode RV-Fabric removes. On a shared log a dropped or misordered event is invisible to the monitor, which keeps emitting "no violation" over a corrupted view (a *silent* failure); RV-Fabric surfaces the breach through event-id gaps, downgrading the verdict to (unknown, incomplete) (Sect. 5).

(3) An evidence-aware verdict model, with a conditional-soundness proposition, attaching a completeness status to every verdict so a degraded stream can no longer produce an unqualified all-clear (Sect. 5). *(4)* An evaluation quantifying the restored properties under crash, clock skew, silence, and overload, including verdict preservation against a fault-free oracle, and replaying two critical-infrastructure datasets through the real engine (Sects. 7, 8).

Novelty and positioning. We claim no new messaging or logging primitive. MQTT, durable stream brokers, transactional outboxes, deduplication, and heartbeats are used exactly as they are, and deliberately so: an operator can deploy RV-Fabric on components already present and already certified in their estate, whereas a novel replication protocol would forfeit precisely that property. What these components do not supply is the link to the verification layer above them. RV-Fabric makes the *completeness of the evidence stream* an explicit part of runtime-verification semantics: each verdict carries a continuity status derived from delivery, ordering, retention, and liveness evidence (Sect. 5), so a corrupted or incomplete stream can no longer produce an unqualified all-clear. A transport layer cannot make this judgement on its own, because it does not know what the monitor required. The accompanying claim is therefore one of *necessity* rather than assembly, and we test it: removing any single mechanism reintroduces one specific silent failure, from 0 to 76 of 1200 timestamp-order violations without G2, a drain time of 0.043 s rising to 40 s without G4, and 1000 of 2000 verdicts silently evicted rather than surfaced without G5 (Table 2); one level up, on the security verdicts themselves, the shared-log baseline preserves one of seven injected incidents while the full fabric preserves all seven, and dropping the tick alone yields two false all-clears that no completeness status can flag (Sect. 7). That is an argument no individual component makes.

2 Background: The Hierarchy and the Shared-Log Prototype

This section states what RV-Fabric inherits: the hierarchical monitoring workload it carries and the single-host shared-log prototype it replaces, both from prior work [15,14]. What RV-Fabric adds begins in Sect. 3.

The three-layer hierarchy. RV-Fabric distributes a hierarchical monitoring workload across three tiers, each seeing a wider slice of the fleet than the one below. *Layer 1 (L1)*, on or beside each device, runs stateless per-event sanity checks (payload size, publish rate, timestamp monotonicity) and emits a compact per-window verdict. *Layer 2 (L2)*, at the gateway, aggregates the L1 verdicts of the devices it serves. *Layer 3 (L3)*, in the backend, evaluates fleet-wide metric first-order temporal-logic (MFOTL [4]) properties such as coordinated cross-device attacks and silent-node failures: first-order quantification with metric bounds (e.g. "two distinct devices exceed a threshold within 2 s") that relate events at several gateways on one ordered timeline and are inexpressible at a single tier [15,14]. The design is engine-agnostic; at L3 we use a hybrid of MonPoly [5] and RTLola [11] from a recent hierarchical RV design [15].

The shared-log prototype and its gaps. In that design's prototype all three layers run on one host over a single shared log, exposing the four failures of Sect. 1 (loss, skew, silence, no flow control). The prototype was reported to drop $\sim$1% of events at ingest [15], but a controlled same-host replay does *not* reproduce this (Sect. 7); the shared log's real exposure is loss under crash or overload, broken per-device ordering under skew, and silence-liveness. Prior work names an event bus as the fix [15,14]; this paper builds that bus and shows it does not suffice on its own, because the monitor must also know when the bus has failed. Figure 2 shows the resulting architecture.

3 The RV-Fabric Architecture

This section derives the transport requirements from each layer's assumptions and presents the fabric that meets them.

From RV assumptions to transport requirements: the design argument. Distributing the hierarchy turns each layer's implicit input assumption into an explicit transport requirement: a verdict is sound only if the monitor sees a *complete* input (at-least-once delivery), in *order* (a gateway-anchored timeline), with *progress* under silence (a periodic tick), read *independently* (per-consumer cursors), and *without loss under load* (flow control). RV-Fabric supplies exactly these five (Sect. 5); each names a way monitoring silently degrades under the conditions an incident produces.

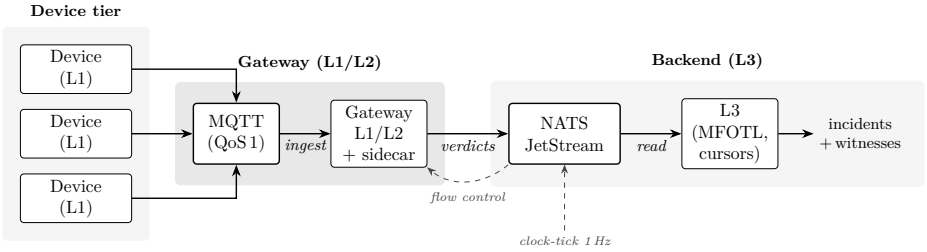

Fig. 2. RV-Fabric: the three-layer RV hierarchy decoupled over a two-broker fabric. Devices publish at QoS 1 to a gateway-partitioned MQTT ingest (logically one broker per gateway; the prototype uses per-gateway topic prefixes on one shared broker, equivalent for routing but without broker-level fault isolation); the gateway runs L1/L2 and a timestamping sidecar and re-publishes verdict streams to a backend NATS JetStream cluster, from which the L3 fleet monitors consume via independent durable cursors. A 1 Hz clock-tick subject drives the time-triggered silent-node monitors, and flow control defers loss under transient overload.

MQTT for device-to-gateway ingest. The device-to-gateway hop is constrained and lossy, so RV-Fabric uses MQTT (Mosquitto, QoS 1) [22,19], which is ubiquitous on IoT and gives at-least-once delivery with explicit PUBACKs; L1 is idempotent per (device, window), so duplicates are harmless.

NATS JetStream for gateway-to-backend delivery. The gateway-to-backend hop must fan a gateway's verdict stream to several independent L3 monitors without a slow one blocking the others. RV-Fabric uses NATS JetStream [26]: per-consumer durable cursors let each L3 monitor read at its own pace, and a dedicated 1 Hz tick publisher writes to a JetStream subject as the heartbeat that time-triggered monitors need. Verdicts are keyed by per-device subject (`gw.`⟨`id`⟩`.verdict`), so each device's stream stays FIFO-ordered while a wildcard subscription (`gw.*.verdict`) fans a whole gateway's devices into one monitor.

Timestamping sidecar. A sidecar co-located with each gateway stamps each event with a trusted, non-decreasing gateway-*ingest* time and orders by it, restoring at the first trust boundary the ordering invariant L3 assumes. Device-reported time is retained only for spoof detection, never for ordering; original device event time is not reconstructed.

Durable relay and deduplication. On receipt the gateway assigns each verdict a persistent event id (⟨gw, device, seq⟩) and appends it to a local `fsync`ed write-ahead outbox *before* forwarding it to JetStream; once JetStream acknowledges the durable publish, an `fsync`ed confirmation cursor advances, so a restart replays only the un-acked suffix, bounding recovery cost. QoS 1 with persistent sessions protects the device-to-broker hop, the outbox the gateway-to-JetStream relay. Backend consumers deduplicate by event id, so replays never double-count,

giving exactly-once monitoring *effect* over an at-least-once transport without distributed transactions [13]. A narrow residual window remains between delivery of the brokered verdict to the relay and its local WAL `fsync`, where the verdict is still only in memory; Sect. 7 injects a crash at each relay stage.

Properties the fabric carries. End-to-end, RV-Fabric carries a hierarchical security-property catalogue [15,14]: single-node checks at L1/L2 and, at L3, cross-device correlation (P3.1–P3.4) plus a time-triggered silent-node property (P3.5). Briefly, P3.1 flags a multi-vector advanced persistent threat (APT) on one device, P3.2 the same attack from ≥ 3 devices, P3.3 an escalation over time, P3.4 a persistent overflow campaign, and P3.5 a device gone silent, each sound only because a specific transport guarantee holds (Sect. 5). The multi-gateway view also lets RV-Fabric express a property the single-gateway prototype cannot observe: P3.6 flags an overflow campaign spanning ≥ 2 distinct gateways,

$$P_{3.6} \;\equiv\; \big(\mathit{cnt} \leftarrow \mathsf{CNT}\, g;\ \mathrm{ONCE}_{[0,30]}\, \mathit{overflow}(g)\big) \;\wedge\; \mathit{cnt} \geq 2, \tag{1}$$

i.e. at least two distinct gateways report an overflow within a 30 s window. A second fabric-enabled property, P3.7, fires when an entire gateway falls silent while the fabric's tick keeps pulsing, distinguishing gateway-level stream loss from the silence of an individual device: $P_{3.7}(g) \;\equiv\; (\mathrm{ONCE}\, \mathit{verdict}(g)) \wedge \neg\, \mathrm{ONCE}_{[0,T_{\mathrm{gw}}]}\, \mathit{verdict}(g)$, evaluated at each tick. P3.7 is a *fabric-enabled availability* property rather than an intrinsically cross-gateway one: it requires only a clock independent of the monitored stream (unlike P3.6, which genuinely needs ≥ 2 gateway identities). That clock is a *backend-resident* tick publisher advancing monitoring time independently of every gateway stream, so silence-liveness is by construction independent of any gateway remaining up, where a per-gateway tick would freeze.

4 Threat Model and Trust Assumptions

RV-Fabric is security-monitoring infrastructure, so its guarantees must be judged against an explicit adversary, not benign-failure assumptions alone. This section states what RV-Fabric defends, what it trusts, and which guarantees hold against which adversary, the threat model the continuity mechanisms of Sect. 5 are designed against.

Asset and adversary goal. The asset RV-Fabric protects is the *monitoring verdict stream*: every security-relevant event durably accepted into the protected path must reach the correlator, in the trusted gateway-ingest order, and the *absence* of events must itself be observable. The IoT payload is out of scope; RV-Fabric defends the evidence about it, not the data. The adversary's goal is to make a genuine attack go *unflagged*, by corrupting the monitor's view (drop/reorder/replay), *blinding* it (silencing a device or gateway), or *exhausting* it (overload that sheds events), the silent-degradation modes of Sect. 1 recast as adversarial objectives.

Table 1. Threat model: for each adversary capability, the RV-Fabric mitigation, the continuity mechanism (G1–G5, Sect. 5) or monitored property (P3.x) it uses, and the residual risk. Entries marked † lie outside the trust boundary (assumed, not enforced).

Adversary capability	**RV-Fabric mitigation**	**Via (G/P)**	**Residual risk**
Network adversary (untrusted links)			
drop / delay events	at-least-once (QoS 1 & outbox); flow control absorbs transient backlog	G1, G5	observable retention exhaustion, then loss
reorder events	monotone gateway timestamps re-anchor the timeline	G2	cross-gateway skew bounded by offset ϵ (Sect. 7)
replay events	event-id dedup (idempotent monitoring effect)	n/a	ID collision / sequence-state loss; ID forgery†
Device adversary			
emit malicious traffic	L1/L2 checks; fleet correlation	P3.1–4	novel behaviour outside the specifications
spoof its clock	sidecar ignores device time for ordering	G2	device time trusted only for the spoof check
go silent (jam / DoS)	backend tick drives silent-node under total silence	G3, P3.5	cause (fault vs. attack) not identified
Gateway adversary			
taken dark	gateway-silence via the backend-resident tick	P3.7	that fleet's in-flight evidence is lost
compromised†	*out of scope*: sidecar integrity trusted	n/a	full loss of that fleet's monitoring soundness

Trusted computing base. RV-Fabric trusts the cloud backend and L3 monitors, the two brokers under their configured durability, and each gateway's timestamping sidecar and durable outbox, a deliberately small TCB on the operator's side. Everything device-side of a gateway, the devices and the network links, is untrusted; link cryptography (broker TLS and authentication) is assumed, so the network adversary can manipulate traffic but cannot forge a broker-authenticated identity.

Adversary treatment and scope. Table 1 maps each capability to its mitigation. The key property is that an adversary who *silences* a link cannot thereby hide the silence: P3.5/P3.7 turn suppression into a positive detection, though evidence may be delayed during a prolonged partition. The residual is behaviour that is malicious yet specification-conforming, the general limit of any specification-

based monitor. A gateway *taken dark* is handled (P3.7 plus outbox replay on recovery); an *actively compromised* one is out of scope, its sidecar and outbox lying in the TCB, so RV-Fabric bounds the blast radius by keeping fleets disjoint (Sect. 6).

5 Continuity Mechanisms and Their Invariants

This section presents the continuity mechanisms RV-Fabric establishes over the two brokers, states each as a broker-contingent invariant tied to a specific shared-log failure, and defines the verdict algebra coupling them to the output.

RV-Fabric provides five continuity mechanisms, which we label as *guarantees* **G1** Delivery, **G2** Order, **G3** Tick, **G4** Isolation, and **G5** Flow (the letter G stands for *guarantee*). We state each as an *invariant* of the fabric, contingent on the durability configuration of the components that supply it (Sect. 6): their value is the explicit correspondence, RV assumption → transport property → shared-log failure closed.

Two carry the most weight. The *clock-tick* is the subtlest: a time-triggered "no device has reported for T_{dev} seconds" property cannot fire from a shared log, since a log with no new lines produces no events to advance the monitor's clock, so a 1 Hz tick supplies that liveness under total silence. *Flow control* only *defers* loss by letting events queue: sustained overload beyond the configured retention capacity, evaluated in Q7, makes exhaustion observable but cannot preserve G1 beyond capacity, so delivery is conditional on capacity.

Guarantees as invariants. Write E_{brk} for messages the ingest broker has durably accepted, E_{gw} for those the gateway receives, E_{wal} for verdicts `fsync`ed to the gateway WAL, and $\hat{E}$ for what the backend L3 monitor observes; each verdict v carries a gateway timestamp $\tau(v)$. The five guarantees are fabric invariants: *(G1)* at-least-once on two protected segments, $E_{\text{brk}} \subseteq E_{\text{gw}}$ (persistent MQTT hop, given durable QoS 1 acceptance under a persistent session) and $E_{\text{wal}} \subseteq \hat{E}$ (gateway–backend relay), so nothing durably accepted is lost, while drops *before* durable acceptance (device crash, device-side queue, pre-WAL window) lie outside G1; *(G2)* monotonicity, if v_1 is delivered before v_2 then $\tau(v_1) \leq \tau(v_2)$ for same-device verdicts; *(G3)* liveness, a 1 Hz tick drives the backend even when $\hat{E} = \emptyset$; *(G4)* isolation, each consumer's cursor never gates the others; and *(G5)* flow control, awaited publishes and retention queue transient overload rather than dropping it. G1–G2 protect and re-order the monitoring view; G3–G5 enable resilient distributed operation.

Evidence-aware verdicts. The mechanisms above bound *when* the input stream is trustworthy; a monitor that keeps emitting a bare "no violation" after the stream is corrupted reproduces the silent failure we set out to remove. We therefore make evidence completeness explicit in the verdict semantics. Following three-valued RV semantics [8], which admit an inconclusive verdict when the observed

trace is insufficient, a verdict is a pair $V = (s, c)$ of a *security* and an *evidence-completeness* component,

$$s \in \{\mathsf{violation},\ \mathsf{no\_violation},\ \mathsf{unknown}\},$$
$$c \in \{\mathsf{sound} \succ \mathsf{degraded} \succ \mathsf{incomplete} \succ \mathsf{unavailable}\},$$

where c is derived from continuity evidence the fabric already tracks: c=sound when every event on the protected segments is accounted for (G1, no gap); degraded past a soft retention high-water mark (G5) before any loss; incomplete when a sequence gap, retention overflow, or a monotonicity violation on the consumed stream (G2) means events are provably missing or misordered; and unavailable when the tick (G3) shows the monitor live but no evidence is arriving. Crucially, unavailable is an evidence-*quality* tag, not a verdict: for a property whose violation *is* the absence of events (the silent-node P3.5 and gateway-silence P3.7), the tick reliably witnesses that absence, so it yields (violation, sound); unavailable applies only to properties that need positive event evidence to decide.

The two components interact by one rule that keeps security and completeness *separate* axes rather than one ad-hoc lattice. For the monotone witness properties formalised in Prop. 1, a *positive* finding backed by an intact local witness is self-evidencing (the witness *is* the evidence) and remains valid under *unrelated* missing evidence, so (violation, c) stands for any c; a *negative* finding is only as good as its input, so a no_violation over non-sound evidence is downgraded, (no_violation, c) $\mapsto$ (unknown, c) for $c \prec$ sound, stopping a corrupted stream from yielding an unqualified all-clear. At L3 a property spanning gateways $g_1, \ldots, g_n$ carries the meet of their evidence, $c_{\mathrm{L3}} = \min_{\preceq}\{c_{g_1}, \ldots, c_{g_n}\}$, so one incomplete gateway makes a fleet-wide all-clear unknown. We implement all four tags over the real JetStream (event-id continuity for gaps, a consumed-stream monotonicity check for order violations, the retention watermark for degraded, tick-versus-evidence for unavailable) and confirm each on a canonical scenario: a normal stream stays sound, an 85% retention high-water mark is degraded, an event-id gap is incomplete (downgrading no_violation to unknown), and a required stream silent under a live tick is unavailable.

Soundness of the verdict algebra. The downgrade rule is not merely a convention: it makes the algebra *sound* for a well-defined and practically dominant class of properties. Call φ a *monotone witness property* if its satisfaction over a window is certified by a finite set of events and preserved under supersets, i.e. for traces $\hat{\sigma} \subseteq \sigma$, $\hat{\sigma} \models \varphi \Rightarrow \sigma \models \varphi$. The counting and multi-vector L3 properties P3.1–P3.4 and P3.6 (existential ONCE-formulas over device or gateway events) are of this class. The absence-based P3.5/P3.7 are dual: G3 gives their liveness while G1/G5 rule out loss-induced false alarms; we treat the monotone class here. Let σ be the durably-accepted event stream for the monitored identities over the window (events that reached the protected path; device-side and pre-WAL losses lie outside G1 and hence outside this result), and $\hat{\sigma} = \mathrm{dedup}(\hat{E})$ the stream L3 observes after event-id deduplication.

Proposition 1 (Conditional soundness of verdicts). *Assume the order guarantee G2 (so $\hat{\sigma}$ is order-faithful) and, for the cross-gateway property P3.6, a bounded cross-gateway skew $\epsilon < W$; the hypothesis $c = \mathsf{sound}$ used below itself presupposes G1, G3, and G5 over the window. For any monotone witness property φ and emitted verdict $V = (s, c)$:* (i) Positive soundness. *If $s = \mathsf{violation}$ then $\sigma \models \varphi$: with the deduplication state intact, no loss or overload manufactures a false violation.* (ii) Negative soundness under completeness. *If $s = \mathsf{no\_violation}$ and $c = \mathsf{sound}$ then $\sigma \not\models \varphi$; a sound all-clear agrees with the fault-free oracle over the accepted stream.* (iii) No silent false all-clear. *If $\sigma \models \varphi$ but the monitor would emit* $\mathsf{no\_violation}$ *(a missed violation), and the loss is observable in the evidence stream, as an event-id discontinuity (G1/G5) or an order violation on the consumed stream (G2), then $c \prec \mathsf{sound}$, so the algebra emits* $(\mathsf{unknown}, c)$ *rather than an unqualified all-clear.*

Proof (sketch). (i) By intact event-id deduplication (exactly-once *effect*), $\hat{\sigma} \subseteq \sigma$: redelivery cannot inflate a count and loss only removes events, while G2 keeps the ONCE-windows aligned (for P3.6, $\epsilon < W$ aligns the cross-gateway window). A satisfying witness $\Omega \subseteq \hat{\sigma}$ gives $\Omega \subseteq \sigma$, hence $\sigma \models \varphi$. *(ii)* $c = \mathsf{sound}$ certifies, over the window, no event-id gap on the protected segments (G1: no delivery loss), backlog below the high-water mark (G5: no eviction), and no order violation (G2), so $\hat{\sigma}$ equals σ restricted to the window's relevant events and $\hat{\sigma} \not\models \varphi \Rightarrow \sigma \not\models \varphi$. *(iii)* Contrapositive of (ii): a missed violation means the accepted witness was lost or misordered in $\hat{\sigma}$; broken event-id continuity, or an order violation on the consumed stream, each set $c = \mathsf{incomplete} \prec \mathsf{sound}$, triggering $(\mathsf{no\_violation}, c) \mapsto (\mathsf{unknown}, c)$. □

The dual absence-based P3.5/P3.7 obtain the analogous guarantee through the tick, which times the silence G1/G5 show to be real. Clause (iii) is *conditional on the loss being observable*, and that condition is necessary rather than incidental: event-id continuity makes delivery loss observable, a monotonicity check on the consumed stream makes reordering observable, and the tick makes silence observable. A transport carrying none of them (the shared log) cannot lower c, so every missed violation there is a silent false all-clear; equally, without the tick a genuine silence is unobservable, the one loss class the algebra cannot flag. The oracle experiment (Q8, Sect. 7) validates all three clauses empirically.

6 Implementation

This section details how the architecture of Sect. 3 is realised: the two brokers, the naming scheme routing events, and the mapping onto the continuum.

Brokers and subjects. Each gateway ingests device telemetry over MQTT (Mosquitto [19]) at QoS 1 on topics `dev/⟨gw⟩/⟨id⟩/evt`; the two-gateway testbed shares one broker with per-gateway topic prefixes (a dedicated broker per gateway is a drop-in alternative), so a gateway consumes only the fleet homed to it. Its L1/L2 consumer re-publishes per-window verdicts to a backend

NATS JetStream [26] stream, routed by *subject*: one subject per device, plus a fleet-wide `fleet/tick` subject carrying the 1 Hz liveness heartbeat.

Monitors. RV-Fabric is deliberately monitor-agnostic: it reuses the L1/L2/L3 monitors of the hierarchical design under study [15,14] unmodified. L1/L2 are lightweight; L3 is the hybrid of MonPoly [5] for metric first-order correlation and RTLola [11] for the time-triggered silent-node property, the latter driven by the fabric's 1 Hz tick subject. Because only the transport beneath them changes, the specification logic is unchanged, anchored to the gateway-ingest timeline, so an operator adopts RV-Fabric without rewriting specifications.

Deployment topology. L1 runs at the edge; each gateway node runs an L1/L2 consumer and a sidecar over a disjoint fleet, so gateways fail independently; a cloud backend runs JetStream and the L3 monitors. The testbed instantiates two gateway nodes of four emulated devices each, and scales out by adding nodes.

Reproducibility. Host. The containerised testbed runs on a single laptop, an Apple M4 Pro MacBook Pro (12 cores: 8 performance + 4 efficiency; 24 GB RAM; macOS 15.3), with Docker 28.1.1. The two-host clock-skew experiment (Sect. 7) instead uses two separate cloud VMs (1 vCPU, Ubuntu 24.04, independent kernels, NTP disabled on one) to obtain real cross-host skew. *Software.* The stack comprises Mosquitto 2 (MQTT), NATS 2 JetStream, the Python gateway/sidecar/relay, and an L3 image that builds MonPoly (OCaml) and RTLola (Rust) from source; emulated devices replay the workload traces. Everything is wired with Docker Compose and a single command brings up the fabric plus a coordinated-attack overlay. Specifications, compose files, and monitor wiring are available as a public artefact.[1]

7 Evaluation on the Controlled Testbed

This section evaluates whether RV-Fabric preserves its five continuity mechanisms under fault injection and at fleet scale, and quantifies the latency and recovery cost of doing so. The mechanisms are the means; what is ultimately under test is evidence completeness, so the sequence closes with Q8, asking whether the security verdicts themselves survive and whether those that do not are at least reported as incomplete rather than as an all-clear.

Testbed and methodology. We deploy RV-Fabric on a multi-gateway testbed: G gateways each serving a share of N emulated devices that replay a documented attack and benign workload [15,14], brokers first colocated and then separated by an emulated wide-area link, all containerised on the host detailed under Reproducibility above. Detection figures are means over five runs; per-event latency (Q2) figures are medians over a paced sample run per link setting, with WAN

[1] `https://github.com/nikos-kekatos/resilient-rv-fabric-edge-iot`

delay injected via Linux `netem`. We ask eight questions, each naming the guarantee (G1–G5) or monitored property (P3.x) it targets: (Q1) *correctness*, does the fabric close the documented gaps (G1–G3); (Q2) *overhead*, the per-event latency added; (Q3) *throughput*, the sustained message rate (G4–G5); (Q4) *detection* of cross-tier, cross-gateway and silent-node attacks (P3.1–P3.7); (Q5) *ablation*, the failure removing each mechanism reintroduces (G1–G5); (Q6) *crash tolerance* at each relay stage (G1); (Q7) *overload horizon* before continuity is lost (G5); and (Q8) *verdict preservation* against a fault-free oracle (G1–G5).

Where the fabric differs (Q1). We compare RV-Fabric against the single-host shared log (the same hierarchy *without* the broker fabric), replaying one fixed 7000-event trace through both at matched load. For steady-state ingest the two are indistinguishable: *both* deliver every event to L1 (0 loss, at 500 and 3000 msg/s, five trials each). The ∼1% ingest loss reported for the earlier prototype [15] did not reproduce, so it is condition-specific and not where the fabric differs. It differs on two properties the evaluated shared-log design lacks. *Ordering*: on the same 1200-verdict stream, device-time ordering leaves 76 out of order under the time-spoof profile, the gateway sidecar 0. *Silence-liveness*: under total silence the shared log yields no events so P3.5 never fires, whereas the 1 Hz tick fires it within one tick of the ∼5 s deadline.

Per-event latency under a wide-area link (Q2). We place the fabric clients behind an emulated wide-area link (Linux `netem`, one-way delay δ per hop) and measure publish-to-consume latency per hop (mean/p95 in ms, paced 400-sample run). Colocated, both hops are sub-millisecond in the mean (0.5/1.0 MQTT, 0.9/1.8 JetStream). They scale differently: the MQTT hop grows as $\approx 3\delta$ because QoS 1's acknowledged handshake crosses the link several times (mean 17/32/60 ms at δ=5/10/20 ms), the JetStream hop as $\approx\delta$ (7/13/23 ms). Even at δ=20 ms, p95 stays under 75 ms, over an order of magnitude below the second-scale L3 detection latency, so transport was not the bottleneck and QoS 1's steeper WAN cost buys at-least-once delivery on the lossy edge hop.

Throughput and fleet scale (Q3). A saturation sweep (one durable consumer, rising producer count) climbs from ∼10.7K to a ∼14K msg/s plateau at zero loss; past the drain rate, offered load queues rather than drops (p95 rising to ∼1 s: flow control defers loss). Emulated fleets of 250/500/1000 devices (≈2 msg/s each) all sustain *zero loss*; at 1000 devices the ≈1930 msg/s aggregate runs at ∼60 ms p95. This validates the message-handling path at fleet scale, not the resource limits of 1000 physical devices; read capacity scales with more L3 consumers.

Detection (Q4). An eight-device, two-gateway workload (mixed overflow/APT/spam/time-spoof/clean) over five 150 s runs fired all six properties P3.1–P3.6 on every run (41.6 ± 2.1 incidents), including the fabric-dependent ones: cross-device **P3.2** and cross-gateway **P3.6** need the backend fleet view and per-consumer cursors (P3.6 is unobservable single-gateway), and time-triggered **P3.5** surfaces only because the tick drives polling under silence.

Taking one gateway dark mid-run, **P3.7** flags gateway-stream loss within T_{gw}=10 s, distinct from a single device going quiet (though not identifying the cause).

Clock-skew resilience of P3.6 (Q4, two independent hosts). We validate P3.6's skew resilience on *two independent machines* (cloud VMs, separate kernels, real network, NTP disabled on gw2 so its clock diverges genuinely). A per-gateway clock-beacon measures the realised offset ϵ against a backend reference, with a zero-skew control at ϵ=0.001 s. Sweeping gw2's injected offset over 0/5/10/20/35 s, the measured ϵ tracks it to within network jitter (0.001/4.75/9.71/19.64/33.76 s), and P3.6 ("$\geq$ 2 gateways within W=30 s") stays sound while $\epsilon < W$ and flips once ϵ exceeds the window, as expected for a windowed property. A real cross-region WAN and correlated multi-gateway failure remain future work (Sect. 10).

Failure mode on mechanism removal (Q5). Each guarantee is characterised by the failure its removal reintroduces (Table 2), rather than by a single scalar. Consumer isolation (G4) we measure directly: a fast L3 consumer drains 2000 verdicts in 0.043 ± 0.001 s (five runs) while a slow peer (20 ms/verdict) on its own durable cursor has processed only 3, a $\sim$940$\times$ isolation against the 40 s a single shared cursor would impose by head-of-line blocking. The other four are quantified elsewhere (Table 2). Q8 then carries each of these transport-level failures one level up, to the security incidents each disabled mechanism causes to be missed or silently mis-reported.

Crash recovery of the durable outbox (Q6). We inject a hard crash (`SIGKILL`-equivalent) into the gateway relay at each of its three stages while it relays 50 verdicts (crash on #25), then restart and read the whole stream through backend deduplication. Crashing *before* the local WAL `fsync` (S1), while the verdict is still only in memory, loses that one in-flight verdict, the residual window of Sect. 3. Crashing *after* it loses nothing: persisted-but-unacknowledged (S2) and acknowledged-but-cursor-not-advanced (S3) both lose 0, S3 re-publishing one duplicate that deduplication collapses to 0 double-counted incidents (exactly-once monitoring *effect*). Recovery is linear in the backlog: a lost cursor forcing a full replay takes 8.8/95/474 ms for 100/1000/5000 entries ($\approx$0.095 ms/entry).

Overload buffer horizon (Q7). Flow control defers loss only while the backend buffer holds; we measure what happens beyond it. Filling a bounded JetStream stream (C=1000, pushed to $2C$ with no consumer) under the two discard policies shows the operational choice sharply: default *discard-old* silently evicts 1000 of 2000 verdicts with 0 publish errors, whereas *discard-new* raises 1000 explicit publish errors, making the same loss *observable*. Crossing the soft high-water mark marks the stream degraded; the first rejected or evicted publication marks it incomplete, consistent with the algebra of Sect. 5 (retaining or retrying the rejected event is future work, Sect. 10). The buffer absorbs an outage of $T_{\text{buffer}} = C/R_{\text{in}}$: at $\approx$1930 msg/s, a million-message buffer covers $\approx$8.6 min.

Table 2. Per-mechanism ablation: the failure each continuity mechanism's removal reintroduces, and the measured effect (with vs. without the mechanism). Numbers are from the experiments cross-referenced in the last column.

Mechanism	Failure on removal	Measured effect (with → without)
G1 Delivery	crash / overload loses un-acked evidence	1 in-flight event lost if the crash precedes WAL `fsync`, 0 after; without the outbox the whole un-acked suffix is at risk (Q6)
G2 Order	metric monitors see a reordered stream	$0 \to 76$ of 1200 timestamp-order violations under time-spoof (Q1)
G3 Tick	silent-node never fires under total silence	P3.5 fires within one tick → 0 pulses / no firing when the tick is absent (Q1)
G4 Isolation	one slow consumer stalls all (head-of-line)	fast drain 0.043 s → 40 s shared cursor (∼940×) (Q5)
G5 Flow	transient overload dropped, not queued	discard-new surfaces every drop → discard-old silently loses 1000/2000 (Q7)

Verdict preservation against a fault-free oracle (Q8). The ablation (Q5) shows that removing each mechanism reintroduces a transport-level failure; this experiment measures the consequence one level up, on the *security verdicts* themselves. We fix a seeded workload that triggers all seven properties P3.1–P3.7 and take the incident set the *unmodified* MonPoly engine (with tick-driven reference detectors for the liveness properties P3.5/P3.7) produces from the fault-free stream as the ground-truth *oracle* $I^\star$ ($|I^\star|$=7). We then inject one seeded fault campaign, replayed as an identical alert sequence under the shared log, the full fabric, and the fabric with each guarantee removed, its faults realised by the measured behaviour of Q6–Q7 and each bound to one incident: a pre-`fsync` crash loses `e1`'s fifth overflow (P3.4's count $5 \to 4$, G1); `c1`'s overflow is transposed after its time-anomaly, breaking the P3.1 window (G2); *discard-old* evicts backlog, dropping the P3.2 botnet count (G5); node `f1` and a third gateway `gw3` go silent at fixed offsets for P3.5/P3.7 (G3); and the P3.6 campaign spans two gateways. The P3.3 escalation on `d1` is a device-local *control* no fault touches, and is the single incident the shared log preserves. Since each incident is hit by at most one fault, the one-miss-per-mechanism structure is a property of the seeded campaign, not per-configuration tailoring; ids, timestamps, and bindings are pinned in the artefact (`exp_oracle.py`). The only variable is which alerts, and in what order, reach the unmodified detectors, so any divergence from $I^\star$ is attributable to the transport. This is a controlled causal experiment across the seven property classes, not a statistically representative benchmark. Each missed incident is labelled by the algebra of Sect. 5 from the state a consumer observes: a miss downgraded to incomplete surfaces as an honest *unknown*, one it cannot flag as a *false all-clear*.

Table 3. Q8: fault-injected verdict preservation against the fault-free oracle ($|I^\star|$=7 incidents spanning P3.1–P3.7). One combined fault campaign (crash, reorder, silence, overload) is replayed at the alert level under each configuration; incident sets are computed by the *unmodified* MonPoly engine plus tick-driven reference detectors. A *false all-clear* is a missed incident the evidence algebra cannot flag: a silent no_violation; a *downgraded* miss is instead surfaced as incomplete.

Configuration	Preserved	Missed	False all-clears	Downgraded to unknown
Baselines				
Shared log + faults	**1/7**	6	**6**	0
RV-Fabric (full)	**7/7**	0	**0**	0
Single-mechanism ablations ($-G_i$)				
−G1 durability	6/7	1	0	1 (eid gap)
−G2 order	6/7	1	0	1 (order viol.)
−G3 tick	5/7	2	2	0
−G4 isolation	7/7	0	0	0 (timeliness)
−G5 flow	6/7	1	0	1 (eid gap)

Table 3 reports the outcome. Every one of the shared log's six misses is a silent false all-clear: it cannot express the gateway-scoped P3.6/P3.7, has no tick for the silent-node P3.5, and drops the crash-, reorder-, and overload-hit incidents with no signal an operator could act on. The full fabric absorbs every fault and so reproduces the oracle by construction; the informative rows are the ablations, where each single-mechanism removal bar the timeliness guarantee G4 reintroduces one class of miss: direct causal evidence that the mechanism, not incidental engineering, preserves the verdict. The central result is the last column. Removing G1, G2, or G5 leaves the evidence signal intact (event ids for the G1/G5 losses, the monotonicity check for the G2 reordering), so the miss is *downgraded* to a flagged unknown rather than reported as a clean all-clear. G3 is the exception: the tick is the very liveness clock the completeness status is computed against, so removing it makes the two silence incidents unflaggable and they revert to false all-clears, the one loss class the algebra cannot surface. G4 preserves the incident *set* but not delivery *latency*, quantified above.

Independent re-execution (reproducibility). Re-running the public implementation on a *fresh single host* with native brokers reproduces the qualitative claims with 95% CIs: 0% loss at 250/500/1000 devices (1997 ± 20 msg/s, $p95$ 121 ± 6 ms at 1000), saturation near 7k msg/s, isolation $\sim$208×, and the crash result *exactly*. Absolute figures are host-dependent; the re-execution establishes reproducibility of the *behaviour*, not of the numbers.

Capacity bounds for scale-out. The testbed is lab-scale, so we state the measured figures as *bounds a larger deployment must respect* rather than as a scale claim. Per durable stream, ingest is bounded by the publish ceiling ($\approx$10.7 K msg/s

single producer, rising to a ≈14 K msg/s plateau at zero loss); past the drain rate the offered load queues rather than drops, so exceeding the bound costs latency ($p95$ ∼1 s) before it costs evidence. A fleet is sized against that ceiling: the 1000-device workload (≈1930 msg/s) uses under a fifth of it, and read capacity scales independently with additional L3 consumers. Tolerance to a backend outage is $T_{\text{buffer}} = C/R_{\text{in}}$, and resuming after a gateway crash costs ≈0.095 ms per un-acked entry (474 ms for a 5000-entry replay), so both buffer capacity and recovery time follow from the fleet rate and the backlog a deployment permits. These bounds are per-stream and per-gateway and therefore compose across gateways; what a single host cannot establish is behaviour under a real cross-region WAN and under correlated multi-gateway partition, which is why the ordering guarantee is validated across two independent machines above (Q4) and why the two remaining threats are named explicitly in Sect. 10.

8 Real-Data Validation: Water-SCADA and IoT/IIoT Datasets

We validate the fabric on real critical-infrastructure data, replaying two openly-published datasets, WUSTL-IIoT-2021 [30] (water-supply SCADA) and TON_IoT [21,1] (IoT/IIoT), through the real MonPoly L3 engine. *Scope.* The question is transport, not detection accuracy: whether real security-event streams can be carried and correlated end-to-end while preserving the RV assumptions of completeness, ordering, and liveness. These are flow logs, not distributed deployments, so they exercise detection through the unmodified engine, not the mechanisms G1–G5; thresholds are fixed a priori and untuned, and as neither dataset records a gateway assignment the cross-gateway property runs over a *synthetic* partition (Sect. 10), each identity hashed to a gateway independent of label and attack class, so it cannot have been tuned for cross-gateway firings.

Because the device-payload predicates of Sect. 7 target application misuse rather than network-flow malware, we add three flow monitors over *failed* connections (Zeek `conn_state=S0` or Argus `DstPkts=0`), applied unchanged to both datasets: **port_scan** (≥ 20 failed destinations in 60 s), **ddos_flood** (≥ 100 connections to one destination in 10 s), and **c2_beacon** (≥ 50 periodic retries, CV<0.5). These feed L3 as predicate-parametric schemas: $P3.2(A)$ fires when ≥ 3 source identities satisfy alert A in the window, $P3.6(A)$ when ≥ 2 gateways do; detection is scored per identity.

Both datasets carry and correlate end-to-end. WUSTL-IIoT-2021 is a water-SCADA Modbus trace (≈1 M Argus flows, 6 of 14 source identities attacking); TON_IoT a UNSW cyber-range trace (461k records, 19 of 11,536 identities attacking over nine classes, stressing correlation over many identities). Per-identity detection (precision/recall, untuned) is 0.83/0.83 (WUSTL) and 0.64/0.47 (TON_IoT); dedicated detectors (e.g. BATADAL [27]) score higher. Driving the *real* MonPoly engine, the coordinated **P3.2** schema fires 2/11 episodes and the cross-gateway **P3.6** 1/168 (WUSTL/TON_IoT), at up to

three coordinated sources across two gateways, validating the fleet properties on labelled multi-source traffic. Application-layer classes (injection, XSS, ransomware, MITM) need content-level specifications and are out of scope. These counts are *episodes* (consecutive satisfaction timepoints collapsed), so they are distinct coordinated events rather than per-timepoint alerts. P3.6's count depends on how the synthetic partition places identities across gateways and should be read as partition-specific; P3.2, quantifying over devices, does not.

9 Related Work

RV-Fabric builds directly on two recent hierarchical RV designs for edge-IoT security: the first introduces the three-layer hierarchy and its hybrid MonPoly/RTLola backend for fleet-wide temporal correlation [15], the second the formal-methods specification of the layered security properties [14]. Both are validated as single-host prototypes reading a shared log, leaving the delivery layer, and thus resilience, out of scope. This paper supplies the layer they lack, carrying the same unmodified hierarchy across the edge-cloud continuum and making evidence completeness part of each verdict.

Measured against the five guarantees, the shared-log RV prototype [15,14] and MQTT-only monitoring lack durable delivery, independent consumers, a silence-aware clock, and trusted ordering, while generic stream processors supply durability and isolation but no RV-specific ordering and no critical-infrastructure threat model (Sect. 4).

Hierarchical and spatial RV. STREL [2] and SaSTL [20] formalise hierarchical and spatially distributed monitoring of cyber-physical systems; the hierarchy RV-Fabric carries reuses that decomposition pattern [15,14]. This line develops the *logic* of multi-tier monitoring; RV-Fabric is orthogonal, addressing how such a hierarchy is *deployed* resiliently across a real edge-cloud network.

Distributed, decentralised, and fault-tolerant RV. RV at scale has been studied at the algorithm level, in stream-based specification languages, and in distributed and decentralised monitoring [18,7], and applied to CPS co-simulation and digital-twin pipelines [28,29]. Closest to us is the fault-tolerant line: Fraigniaud et al. characterise the opinions a fault-tolerant distributed monitor needs [12], Bonakdarpour et al. give crash-resilient decentralised algorithms [9], and Basin et al. monitor MFOTL over *out-of-order* streams [6]. That work makes the monitoring *logic* fault- or reordering-tolerant while abstracting the transport; RV-Fabric sits below it, making the delivery layer's loss, ordering, and liveness properties first-class and measurable, so the stream those algorithms consume is the stream their correctness assumes.

Cloud-native messaging and critical-infrastructure security. MQTT [22,19], NATS [26], and Kafka [17] are the standard cloud-native telemetry transports;

RV-Fabric repurposes this layer for *verification*, requiring the five guarantees of Sect. 5 rather than best-effort delivery. Orthogonally, IoT security monitoring spans surveys [10], ML intrusion detection [23], and attack-path analysis across critical infrastructures [25] over the edge-cloud continuum [24]. The tiered pattern recurs across sectors, including resilience architectures for electrical power and energy systems [16], so the delivery-layer guarantees are sector-independent; RV-Fabric contributes that verification-grade delivery layer, not the detection logic.

10 Limitations and Threats to Validity

Testbed scope. The throughput, crash-recovery, and detection experiments run on a single containerised host. That establishes that each mechanism holds its stated invariant under injected faults and that removing it reintroduces one specific failure; it does not establish the absolute throughput or latency of a production deployment, whose figures are host- and network-dependent (the fresh-host re-execution reproduces the behaviour, not the numbers). We therefore report capacity as bounds a scale-out must respect (Sect. 7) rather than as a scale claim, and crash tests cover one gateway's relay path, not correlated multi-gateway failure. The evaluation is not uniformly single-host: P3.6's skew resilience is measured across *two independent machines* with separate kernels and a real network (Q4) against an $\epsilon \approx 0$ control, so the ordering guarantee is not an artefact of shared-host scheduling. The immediate next experiments are multi-host operation at fleet scale, a real cross-region WAN (our hosts are same-region), and correlated multi-gateway partition. *Clock-tick source.* The tick is backend-resident, so silence-liveness survives the loss of every gateway (verified with both killed); the residual dependency is the backend, which a production deployment should replicate. *Trusted gateway boundary.* The sidecar and outbox lie inside the trusted computing base, so a compromised gateway that emits plausible heartbeats while suppressing or forging evidence is out of scope: RV-Fabric detects a gateway that goes *silent* (P3.7) but not one that lies. Narrowing this, e.g. with hash-linked event-sequence commitments or signed evidence receipts that make suppression tamper-evident, is a natural next step. *Real-data study.* It validates that the fleet properties fire end-to-end on real traces, not the continuity mechanisms; the cross-gateway result uses a synthetic gateway partition and detection is untuned. The deterministic crash/ordering counts (Q1, Q6) reproduce exactly; stochastic figures carry 95% intervals in Sect. 7.

11 Conclusion

RV-Fabric turns a single-host, shared-log RV prototype into a resilient delivery layer for security monitoring of critical edge-IoT deployments: five broker-contingent continuity mechanisms add crash- and overload-resilient delivery and close the observed ordering and silence-liveness gaps, at an overhead small relative to second-scale detection. Beyond resilient transport, RV-Fabric makes

evidence completeness part of runtime-verification semantics, so faults and overload can no longer silently turn missing evidence into a false all-clear: against a fault-free oracle through the real MonPoly engine, the full fabric preserves all seven injected incidents where the shared log preserves one and silently misreports the other six. This explicit, empirically validated link between transport continuity and verdict trustworthiness is, we argue, the missing resilience layer between mature hierarchical RV logic and its deployment on real critical infrastructure.

Acknowledgements. This work has received funding from the European Union's Digital Europe Programme under grant agreement No 101190251 (CYBERGUARD).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A., Anwar, A.: TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. IEEE Access **8**, 165130–165150 (2020)
2. Bartocci, E., Bortolussi, L., Loreti, M., Nenzi, L.: Monitoring mobile and spatially distributed cyber-physical systems. In: 15th ACM-IEEE Int. Conf. on Formal Methods and Models for System Design (MEMOCODE) (2017)
3. Bartocci, E., Falcone, Y., Francalanza, A., Reger, G.: Introduction to runtime verification. In: Lectures on Runtime Verification. Springer (2018)
4. Basin, D., Klaedtke, F., Marinovic, S., Zălinescu, E.: Monitoring metric first-order temporal properties. Journal of the ACM **62**(2), 15:1–15:45 (2015)
5. Basin, D., Klaedtke, F., Müller, S., Zălinescu, E.: MonPoly: Monitoring usage-control policies. In: Runtime Verification (RV) (2011)
6. Basin, D., Klaedtke, F., Zălinescu, E.: Runtime verification of temporal properties over out-of-order data streams. In: Computer Aided Verification (CAV) (2017)
7. Bauer, A., Falcone, Y.: Decentralised LTL monitoring. In: Formal Methods (FM) (2012)
8. Bauer, A., Leucker, M., Schallhart, C.: Runtime verification for LTL and TLTL. ACM Trans. Softw. Eng. Methodol. **20**(4) (2011)
9. Bonakdarpour, B., Fraigniaud, P., Rajsbaum, S., Rosenblueth, D.A., Travers, C.: Decentralized asynchronous crash-resilient runtime verification. In: Concurrency Theory (CONCUR) (2016)
10. Cavalli, A.R., Mallouli, W., Montes de Oca, E., Rivera, D.: IoT security monitoring tools and models. In: Springer Handbook of Internet of Things. Springer (2024)
11. Faymonville, P., Finkbeiner, B., Schledjewski, M., Schwenger, M., Stenger, M., Tentrup, L., Torfah, H.: StreamLAB: Stream-based monitoring of cyber-physical systems. In: Computer Aided Verification (CAV) (2019)
12. Fraigniaud, P., Rajsbaum, S., Travers, C.: On the number of opinions needed for fault-tolerant run-time monitoring in distributed systems. In: Runtime Verification (RV) (2014)

13. Helland, P.: Life beyond distributed transactions: an apostate's opinion. In: Conf. on Innovative Data Systems Research (CIDR) (2007)
14. Kekatos, N., Chintri, M., Katsaros, P., Lekidis, A., Nianios, T., Seitoglou, I., Basagiannis, S.: Hierarchical security monitoring for edge-IoT: A formal methods approach. In: IEEE Int. Conf. on Cyber Security and Resilience (CSR), CRE Workshop (2026), to appear
15. Kekatos, N., Chintri, M., Katsaros, P., Lekidis, A., Nianios, T., Seitoglou, I., Temperekidis, A., Basagiannis, S.: Hybrid hierarchical runtime verification for edge-IoT security: Combining MonPoly and RTLola. In: Availability, Reliability and Security (ARES), EU Projects Symposium Workshops. pp. 222–240. Springer (2027)
16. Kekatos, N., Koutidis, G., Antonakopoulos, K., Basagiannis, S., Katsikas, S., Kavallieratos, G., Lekidis, A., Nianios, T., Papageorgiou, E.: RESILAGENT: A three-pillar architecture for cyber-physical resilience in electrical power and energy systems. In: IEEE Int. Conf. on Cyber Security and Resilience (CSR) (2026), to appear
17. Kreps, J., Narkhede, N., Rao, J.: Kafka: a distributed messaging system for log processing. In: NetDB (2011)
18. Leucker, M., Schallhart, C.: A brief account of runtime verification. J. Log. Algebr. Program. **78**(5) (2009)
19. Light, R.A.: Mosquitto: server and client implementation of the MQTT protocol. Journal of Open Source Software **2**(13) (2017)
20. Ma, M., Bartocci, E., Lifland, E., Stankovic, J.A., Feng, L.: SaSTL: Spatial aggregation signal temporal logic for runtime monitoring in smart cities. In: Cyber-Physical Systems (ICCPS) (2020)
21. Moustafa, N.: A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. Sustainable Cities and Society **72**, 102994 (2021)
22. OASIS: MQTT version 5.0. Tech. rep., OASIS Standard (2019)
23. Santhosh Kumar, S.V.N., Selvi, M., Kannan, A.: A comprehensive survey on machine learning-based intrusion detection systems for secure communication in internet of things. Comput. Intell. Neurosci. (2023)
24. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. IEEE Internet Things J. **3**(5) (2016)
25. Stellios, I., Kotzanikolaou, P., Psarakis, M., Alcaraz, C., Lopez, J.: A survey of IoT-enabled cyberattacks: Assessing attack paths to critical infrastructures and services. IEEE Commun. Surveys Tuts. **20**(4) (2018)
26. Synadia / CNCF: NATS and JetStream. `https://nats.io` (2024)
27. Taormina, R., Galelli, S., Tippenhauer, N.O., Salomons, E., Ostfeld, A., et al.: The battle of the attack detection algorithms: Disclosing cyber attacks on water distribution networks. J. Water Resour. Plan. Manage. **144**(8) (2018)
28. Temperekidis, A., Kekatos, N., Katsaros, P.: Runtime verification for FMI-based co-simulation. In: Runtime Verification (RV). Springer (2022)
29. Temperekidis, A., Kekatos, N., Katsaros, P., He, W., Bensalem, S., AbdElSabour, H., AbdElSalam, M., Salem, A.: Towards a digital twin architecture with formal analysis capabilities for learning-enabled autonomous systems. In: Modelling and Simulation for Autonomous Systems (MESAS). LNCS, vol. 13866. Springer (2023)
30. Zolanvari, M., Teixeira, M.A., Gupta, L., Khan, K.M., Jain, R.: WUSTL-IIoT-2021 dataset for IIoT cybersecurity research. `https://www.cse.wustl.edu/~jain/iiot2/` (2021)